

SaG-Prescribe: Unifying Diagnostic Pathways with Counterfactual Refactoring for Automated Code Review and Software Quality Evaluation in Distributed Publish–Subscribe Systems

Author Information Redacted for Double-Blind Review

Special Issue: Intelligent Techniques for Automated Code Review and Software Quality Evaluation
Automated Software Engineering (Springer)

August 30, 2026

Abstract

Automated code review and software quality evaluation are vital for modern software engineering, where systems evolve rapidly and continuous integration demands proactive quality assurance. However, conventional quality assurance practices rely primarily on modular static code analysis (SCA) operating at the file or class level. In distributed publish–subscribe architectures (ROS 2, DDS, MQTT), loose coupling among producers and consumers obscures indirect dependencies, creating an **Architecture-Code Gap**: a codebase may exhibit clean unit-level metrics while harboring severe architectural technical debt, such as single points of failure, co-located deployment bottlenecks, and transport contract mismatches. Furthermore, existing structural diagnostic frameworks operate open-loop—ranking components by scalar criticality without identifying the underlying architectural pathology or verifying remediation actions.

To address these challenges, we introduce **SaG-Prescribe**, an intelligent static system analysis framework that brings a deterministic **Diagnostic Pathway** to the forefront of automated code review and software quality evaluation. The framework unifies two complementary mechanisms: (1) **Explainable Diagnostic Pathway & Technical Debt Analysis**: It extracts a multi-layer heterogeneous dependency graph from Architecture-as-Code descriptors, synthesizes code-level SCA metrics into a **Code Quality Penalty (CQP)**, and computes a hierarchical **Reliability–Maintainability (RM)** quality attribution model grounded in ISO/IEC 25010 and ISO/IEC 25019 Quality-in-Use standards. It drives a catalog of nineteen formal publish–subscribe anti-patterns and bad smells with adaptive box-plot thresholds, providing explainable review decisions mapped to specific engineering roles (Reliability Engineers, SREs, Software Architects). (2) **Prescriptive Refactoring with Counterfactual Verification**: It compiles diagnostic findings into three graph-mutation refactoring operators (logical topic splitting, physical anti-affinity reallocation, and transport QoS contract hardening). Crucially, every candidate refactoring is independently verified on a sandboxed counterfactual graph against a discrete-event cascade simulator, admitting only edits whose measured risk reduction exceeds simulator seed noise across a sweep of propagation thresholds ($\overline{\Delta I} > \kappa\sigma_{\text{seed}}$).

We evaluate SaG-Prescribe across eight benchmark scenarios spanning autonomous vehicles, IoT smart cities, financial trading, healthcare, microservices, and hyper-scale enterprise systems, reporting empirical findings with full methodological transparency. On the diagnostic side, composite criticality correlates moderately with cascade impact (mean Spearman $\rho = 0.485$, app layer), highlighting that scalar ranking is out-performed by degree centrality ($\rho = 0.519$) and that the anti-pattern catalog functions as a highly sensitive screen (mean recall 0.942, precision 0.253, implicating 94.8% of components) requiring threshold recalibration and delta-aware gating. We also document a Simpson’s paradox pooling effect across heterogeneous node types ($\rho = 0.028$ pooled vs. $\rho = 0.503$ on applications). On

the prescriptive side, counterfactual verification admits 1128 of 1589 candidate refactorings (71.0%), with anti-affinity reallocation demonstrating the highest survival rate (77.7%), followed by transport QoS hardening (58.0%) and topic splitting (49.7%). Every scenario achieves a statistically significant net reduction in System Risk Index (Wilcoxon $W = 0, p = 0.0156, n = 7$). The diagnostic review gate executes in 0.01–20.98 s, confirming its feasibility as a real-time, pre-commit automated code review check in CI/CD pipelines.

Keywords: Automated code review, software quality evaluation, diagnostic pathway, architectural anti-patterns, bad smells, technical debt analysis, refactoring recommendation, publish–subscribe middleware, CI/CD quality gates, counterfactual verification.

1 Introduction

1.1 Context and Motivation: The Need for Architectural Code Review

Automated code review and software quality evaluation have become indispensable pillars of modern software engineering [1]. As distributed software systems scale in complexity and release cycles compress under continuous integration and continuous delivery (CI/CD) regimes, automated tools must detect regressions, evaluate non-functional requirements, and recommend actionable remediations before defective changes reach production fabrics.

Distributed publish–subscribe middleware frameworks—including the Robot Operating System (ROS 2), the Object Management Group’s Data Distribution Service (DDS), and MQTT—form the communication infrastructure of contemporary microservices, IoT smart cities, and safety-critical cyber-physical systems [2, 3]. These frameworks decouple message producers and consumers across space, time, and synchronization via topic-based publish–subscribe channels and message brokers. However, this architectural decoupling introduces an insidious challenge for automated quality assurance: it obscures the end-to-end dependency structure of the system. Failures, buffer overflows, and latency spikes propagate along indirect topological pathways that are completely invisible when analyzing source code files in isolation.

1.2 Two Open Gaps: The Architecture-Code Gap and Open-Loop Review

Quality evaluation in continuous integration has historically relied on Static Code Analysis (SCA) platforms (e.g., SonarQube, SpotBugs, ESLint). This creates a fundamental **Architecture-Code Gap**:

1. **The Architecture-Code Gap:** A distributed system can achieve pristine code quality within every individual component—clean class hierarchies, zero file-level code smells, and low cyclomatic complexity—yet remain brittle at the architectural topology level. Vulnerabilities such as single points of failure (SPOFs), severe topic fan-out bottlenecks, co-located deployment risks, and incompatible Quality of Service (QoS) transport contracts cannot be detected by examining source files in isolation. Shifting structural dependability checks left into the development lifecycle demands a transition from traditional Static Code Analysis to *Static System Analysis (SSA)*.
2. **The Unnamed-Pathology and Open-Loop Review Gap:** In object-oriented programming, mature catalogs of bad smells and anti-patterns (e.g., God Class, Feature Envy, Shotgun Surgery) provide developers with a shared vocabulary, testable detection rules, and proven refactoring patterns [4–6]. Microservices research has similarly begun cataloging smells (e.g., distributed monoliths, cyclic dependencies, shared databases) [7]. In contrast, *publish–subscribe architectures lack an equivalent formal anti-pattern catalog*. Consequently, architectural flaws are discovered reactively through production cascade postmortems rather than proactively during code review. Moreover, existing topology-aware diagnostic frameworks typically operate *open-loop*: they compute abstract numerical centrality

scores or black-box failure-impact predictions without identifying the specific architectural pathology at fault or synthesizing verified, actionable refactoring guidance [8, 9].

1.3 The Diagnostic Pathway: Transparent, Explainable Software Quality Evaluation

To address these gaps, this paper brings the **Diagnostic Pathway** to the forefront of automated code review and software quality evaluation in distributed systems. We present **SaG-Prescribe**, a comprehensive framework that integrates deterministic diagnostic quality attribution, anti-pattern detection, and closed-loop prescriptive refactoring within continuous integration workflows.

SaG-Prescribe decouples quality assurance into two principled, cooperative pathways:

- **The Diagnostic Pathway (Deterministic Software Quality Evaluation & Attribution):** The Diagnostic Pathway operates statically on Architecture-as-Code descriptors. It constructs a multi-layer heterogeneous dependency graph (G_{analysis}) encompassing applications, libraries, topics, brokers, and physical hosting nodes. It bridges code-level and architecture-level quality by synthesizing modular SCA metrics into a **Code Quality Penalty (CQP)**. It then applies a hierarchical **Reliability–Maintainability (RM)** attribution model grounded in the **ISO/IEC 25010** (Product Quality) and **ISO/IEC 25019** (Quality-in-Use) standards [10, 11]. Reliability decomposes into **Fault Tolerance** (FT , error propagation reach) and **Availability** (A , structural single-point-of-failure exposure), while **Maintainability** (M) quantifies coupling complexity and change ripple risk. The diagnostic pathway maps structural technical debt directly to stakeholder harm categories ($\mathbf{h}_{\text{QiU}}$) and routes explainable findings to specific engineering roles (Reliability Engineers, DevOps/SREs, and Software Architects). Over these metric vectors, it executes a catalog of nineteen formal publish–subscribe anti-patterns using adaptive box-plot thresholds.
- **The Counterfactual Prescriptive Pathway (Closed-Loop Refactoring Verification):** Rather than offering open-loop suggestions, the prescriptive engine compiles diagnostic findings into three typed graph-mutation operators: logical topic splitting, physical anti-affinity reallocation, and transport QoS contract hardening. It subjects every candidate refactoring to **counterfactual verification** on an in-memory sandbox graph (**MemoryRepository**), simulating failure cascades across multiple seeds and propagation thresholds. An edit is admitted if and only if its measured impact reduction exceeds the simulator’s seed noise at every threshold ($\overline{\Delta I} > \kappa\sigma_{\text{seed}}$), and the composite policy is gated on a net positive System Risk Index delta ($\Delta\text{SRI} > 0$).

1.4 Contributions

This paper makes the following contributions to automated software quality assurance and code review:

1. **A Formal Diagnostic Pathway for Pub-Sub Software Quality Evaluation (Section 3):** We formalize a multi-level diagnostic model that ingests file-level SCA metrics via the Code Quality Penalty (CQP), derives multi-layer dependency projections (G_{analysis}), and computes a hierarchical Reliability–Maintainability (RM) attribution model grounded in ISO/IEC 25010 and ISO/IEC 25019 Quality-in-Use standards.
2. **A Comprehensive Catalog of 19 Publish–Subscribe Anti-Patterns and Bad Smells (Section 4):** We formalize nineteen architectural anti-patterns and smells as topological signatures over a 16-element structural metric vector with adaptive box-plot thresholds, providing explainable review feedback for automated review bots.

- 83 **3. A Prescriptive Refactoring Pipeline with Counterfactual Verification (Sec-**
84 **tions 5–6):** We introduce a rule-based refactoring compiler translating diagnostic findings
85 into three graph-mutation operators (topic splitting, anti-affinity reallocation, QoS harden-
86 ing), verified against a discrete-event cascade simulator using a two-level margin-gating
87 criterion. We disclose the exact automation footprint (5 of 19 patterns automated, 14
88 advisory).
- 89 **4. An Automated Code Review Bot and CI/CD Quality Gate (Section 7):** We
90 operationalize the Diagnostic Pathway as a sub-second pre-merge quality gate featuring
91 standardized exit codes (0/1/2) and formalize delta-aware merge-base regression semantics.
- 92 **5. Empirical Evaluation and Methodological Findings (Sections 8–9):** Evaluated
93 across eight benchmark scenarios (12 to 520 components), we report: (a) diagnostic
94 detection achieves mean Spearman $\rho = 0.485$ against simulated cascade impact, functioning
95 as a sensitive screen (recall = 0.942, precision = 0.253, implicating 94.8% of components)
96 and exhibiting a Simpson’s paradox pooling effect ($\rho = 0.028$ pooled vs. 0.503 on
97 applications); (b) counterfactual verification admits 1128 of 1589 candidates (71.0%),
98 with anti-affinity reallocation demonstrating 77.7% survival, and every scenario achieving
99 statistically significant risk reduction (Wilcoxon $W = 0, p = 0.0156, n = 7$); (c) diagnostic
100 review gating executes in 0.01–20.98 s, confirming real-time CI/CD viability.

101 1.5 Relationship to Prior Work

102 This submission builds upon the foundational graph schema and simulation infrastructure
103 introduced in our companion work [12], while contributing novel, unshared capabilities:

- 104 • Companion manuscript [12] introduces the heterogeneous graph schema, simulation kernels,
105 and a learned Graph Neural Network (GNN) for failure-impact prediction.
- 106 • This paper contributes the **Diagnostic Pathway** for automated code review, the Code
107 Quality Penalty (CQP), the 19-pattern pub-sub anti-pattern catalog, the prescriptive
108 refactoring compiler, the counterfactual verification engine, the CI/CD quality gate, and
109 the comprehensive empirical evaluation of detection and prescription.
- 110 • Detection figures previously reported from preliminary conference workshops are re-
111 measured here with strict artifact reproducibility, reporting negative results and harness
112 corrections with complete scientific candor.

113 1.6 Organization

114 Section 2 surveys related work. Section 3 formalizes the Diagnostic Pathway and software quality
115 model. Section 4 presents the anti-pattern catalog. Section 5 defines the closed-loop optimization
116 objective. Section 6 details the prescriptive pipeline. Section 7 describes CI/CD integration.
117 Sections 8 and 9 present the experimental design and results. Section 10 discusses implications
118 and threats to validity, and Section 11 concludes.

119 2 Background and Related Work

120 2.1 Publish–Subscribe Middleware Dependability

121 Message-oriented middleware research has historically focused on protocol correctness, broker
122 replication, network load balancing, and runtime QoS verification. Classical broker fault-tolerance
123 techniques replicate or partition broker processes to withstand crash faults [13, 14], while log-
124 centric architectures such as Apache Kafka provide distributed commit-log durability [15]. In

125 DDS and ROS 2 ecosystems, recent studies investigate the probabilistic latency characteristics of
126 QoS-driven retransmission protocols [2] and static verification of interdependent QoS policies [3].
127 While Chaos Engineering injects runtime faults into active deployments, it introduces operational
128 hazards and occurs late in the software lifecycle. SaG-Prescribe shifts resilience analysis left into
129 CI/CD, evaluating Architecture-as-Code descriptors statically before deployment.

130 2.2 Automated Code Review and Static System Analysis

131 Automated code review tools traditionally focus on lexical syntax, coding conventions, and
132 localized security vulnerabilities via AST parsing and data-flow analysis (e.g., SonarQube,
133 SpotBugs). Recent research has advanced automated code review using deep learning and large
134 language models for review comment generation and code change assessment [1, 16]. However,
135 these approaches remain confined to source-file boundaries. They cannot infer system-wide
136 topological cascades resulting from decentralized pub-sub message routing. SaG-Prescribe
137 introduces *Static System Analysis (SSA)*, augmenting file-level code quality metrics with global
138 architectural graph models to review structural and middleware-level design flaws.

139 2.3 Software Quality Models and Explainable Diagnostic Pathways

140 Software quality models, standardized in ISO/IEC 25010 (Product Quality) and ISO/IEC
141 25019 (Quality-in-Use), provide structured taxonomies decomposing quality into maintainability,
142 reliability, and usability [10, 11]. While these standards establish top-down quality characteristics,
143 mapping them to concrete, automated measurements in distributed architectures remains
144 challenging. Classical software analytics often collapse multiple structural metrics into a
145 single opaque score or rely on black-box machine learning classifiers that lack explanatory
146 transparency [17]. In automated code review, developers reject opaque recommendations without
147 clear causal rationale. The Diagnostic Pathway in SaG-Prescribe implements a transparent,
148 deterministic attribution hierarchy: it decomposes structural and code metrics into explicit
149 Reliability (Fault Tolerance, Availability) and Maintainability dimensions, mapping technical
150 debt directly to stakeholder harm vectors.

151 2.4 Anti-Pattern and Code Smell Catalogs

152 In object-oriented software engineering, Fowler’s refactoring catalog [4] and Brown et al.’s
153 AntiPatterns taxonomy [5] formalized bad smells (e.g., God Class, Feature Envy) alongside
154 remediation strategies. Suryanarayana et al. systematized design smells using formal structural
155 metrics [6]. In distributed systems, Richardson cataloged microservices design patterns [18],
156 while Taibi et al. established empirical taxonomies of microservices anti-patterns (e.g., Cyclic
157 Dependency, Shared Database, Distributed Monolith) [7]. Our catalog (Section 4) adopts this
158 proven specification structure—formal topological rule, affected quality dimension, and concrete
159 remediation—while addressing the unique structural anomalies of publish–subscribe middleware
160 (e.g., broker saturation, topic fan-out explosion, QoS policy mismatches).

161 2.5 Refactoring Recommendation and Technical Debt Management

162 Automated refactoring recommendation has been investigated across multiple paradigms:

- 163 1. *Metric- and Rule-Based Recommenders*: Rule-driven engines identify code smells and
164 suggest deterministic refactorings [4, 6].
- 165 2. *Search-Based Software Engineering (SBSE)*: Multi-objective optimization algorithms (e.g.,
166 NSGA-II) search architectural trade-off spaces to optimize coupling and cohesion [8, 9, 19].
167 However, traditional SBSE operates open-loop, generating candidate architectures without
168 empirically verifying resilience gains against simulated failure cascades.

3. *Machine Learning-Based Refactoring*: Supervised and self-affirming models predict refactoring opportunities from historical commit data and code quality trajectories [20–22].

4. *LLM-Assisted Refactoring*: Recent foundation models assist in automated code transformation and refactoring explanation [23–25].

SaG-Prescribe differs fundamentally in scope and verification: its scope is the global publish–subscribe topology, and its verification model is strictly closed-loop—every candidate refactoring is measured on a counterfactual sandbox graph against a cascade failure simulator before being presented to the architect.

2.6 Structural Criticality and Graph Centrality

Graph-theoretic analysis has long utilized centrality indices—betweenness [26], PageRank, closeness, and articulation points—to identify critical vertices. Bakhtin et al. applied network centralities to microservice call graphs [27], while complex network studies demonstrate scale-free, hub-dominated characteristics in software dependency graphs [28]. Baldwin and Clark’s design structure matrices [29] and combinatorial network reliability theory [30] provide formal foundations for topological fault tolerance. While theoretical literature posits that multi-dimensional metrics should outperform simple centrality on heterogeneous graphs, our empirical findings (Section 9.1) reveal that scalar degree centrality remains a formidable baseline, reinforcing the necessity of empirical verification over theoretical assertion.

3 System Model and The Multi-Level Diagnostic Pathway

3.1 Heterogeneous Graph Formulation

A distributed publish–subscribe system is modeled as a typed, weighted, directed multigraph:

$$G = (V, E, \tau_V, \tau_E, w_E, w_V) \quad (1)$$

where the vertex type mapping $\tau_V : V \rightarrow T_V$ partitions components into five distinct semantic classes:

$$T_V = \{\text{Application}, \text{Library}, \text{Topic}, \text{Broker}, \text{Node}\} \quad (2)$$

- **Application** (V_{app}): Active execution processes (e.g., ROS 2 nodes, microservices) that publish or subscribe to data channels.
- **Library** (V_{lib}): Shared software libraries linked by applications.
- **Topic** (V_{topic}): Named communication channels routing message streams.
- **Broker** (V_{broker}): Message routing intermediaries (e.g., MQTT brokers, DDS participants).
- **Node** (V_{node}): Physical or virtual compute hosts providing computational substrate.

The edge type mapping $\tau_E : E \rightarrow T_E$ assigns each link to an architectural relationship:

$$T_E = \{\text{PUBLISHES_TO}, \text{SUBSCRIBES_TO}, \text{ROUTES}, \text{RUNS_ON}, \text{CONNECTS_TO}, \text{USES}\} \quad (3)$$

3.2 Derived Dependencies: The DEPENDS_ON Projection

To expose indirect failure propagation channels hidden behind asynchronous pub-sub decoupling, the Diagnostic Pathway derives explicit DEPENDS_ON relations (G_{analysis}) via formal projection rules (directed as dependent $\rightarrow$ dependency):

- **Application-to-Application:** Formed when application A_1 subscribes to a topic T published by application A_2 : $(A_1 \rightarrow A_2) \in E_{\text{depends}}$.
- **Application-to-Broker:** Formed when application A relies on broker B routing topic T : $(A \rightarrow B) \in E_{\text{depends}}$.
- **Application-to-Library:** Formed when application A uses shared library L : $(A \rightarrow L) \in E_{\text{depends}}$, capturing simultaneous blast radius if L experiences a defect.
- **Broker-to-Broker / Node Colocation:** Captures shared-fate host vulnerabilities when multiple brokers or critical services share physical node N .

The projection is organized across four architectural layers:

1. **Application Layer (app):** $V_{\text{app}} \cup V_{\text{lib}}$, focusing on business logic dependencies.
2. **Infrastructure Layer (infra):** V_{node} , capturing compute host topology.
3. **Middleware Layer (mw):** $V_{\text{app}} \cup V_{\text{broker}}$, capturing message mediation.
4. **System Layer (system):** All five vertex types, capturing global cross-layer coupling.

3.3 The Code Quality Penalty (CQP): Bridging SCA and Architecture

To bridge source-code health with architectural risk during automated review, the Diagnostic Pathway ingests file-level static code analysis (SCA) metrics: Lines of Code (`cm_total_loc`), Weighted Methods per Class (`cm_avg_wmc`), Lack of Cohesion in Methods (`cm_avg_lcom`), and the SonarQube Technical Debt Ratio (`sqale_debt_ratio`).

These metrics are rank-normalized and synthesized into a per-component **Code Quality Penalty (CQP)** for Application and Library vertices:

$$\begin{aligned} \text{CQP}(v) = & 0.10 \text{loc_norm}(v) + 0.35 \text{complexity_norm}(v) \\ & + 0.30 \text{instability_code}(v) + 0.25 \text{lcom_norm}(v) \end{aligned} \quad (4)$$

$\text{CQP}(v) \in [0, 1]$ provides an explicit, quantitative bridge from source-code technical debt to architectural criticality, entering directly into the Maintainability dimension of the RM model.

3.4 Hierarchical Reliability–Maintainability (RM) Quality Attribution

Grounded in **ISO/IEC 25010** (Product Quality) and **ISO/IEC 25019** (Quality-in-Use) [10,11], the Diagnostic Pathway decomposes structural quality into two primary dimensions—**Reliability** and **Maintainability**—providing explainable, role-specific diagnostics (Table 1).

Formal Quality Definitions.

- **Definition D1 (Component Criticality):** Let $G_l = (V_l, E_l)$ be a layer-projected graph. Component criticality $\text{crit}_l : V_l \rightarrow [0, 1]^2 \times [0, 1]$ maps vertex v to metric vector $\mathbf{s}(v) = [R(v), M(v)]^T$ and composite score $Q(v)$, estimating Quality-in-Use degradation.
- **Definition D2 (Relationship Criticality):** Let $e = (u, v) \in E_l$ be a dependency edge. Relationship criticality $\text{crit}_l(e) \in [0, 1]$ estimates Quality-in-Use loss resulting from link disruption.

Table 1: RM Quality Dimensions, Stakeholder Roles, and ISO/IEC 25019 Quality-in-Use Mapping.

Dim.	Sub-Characteristic	Engineering Role	High Score Meaning	ISO 25019 Harm
R	Fault Tolerance (FT)	Reliability Engineer	Cascades deeply; large blast radius	Indirect Harm
	Availability (A)	DevOps / SRE	Single point of failure; partition	Primary Task Loss
M	Structural Coupling	Software Architect	High change ripple; code-level debt	Maintainer Effort

• **Definition D3 (Consequence Metric):** Criticality measures structural consequence given failure. Failure likelihood must be supplied externally from operational MTTF telemetry.

• **Definition D4 (Relative Distribution):** Criticality classifications are relative to the score distribution of system S at layer l .

Mathematical Formulation of Quality Dimensions.

1. **Fault Tolerance (FT):** Evaluates cascade reach using Reverse PageRank (RPR) and fan-out concentration:

$$FT(v) = f(\text{RPR}(v), \text{DG}_{\text{in}}(v), \text{MPCI}(v), \text{FOC}(v)) \quad (5)$$

2. **Availability (A):** Evaluates single-point-of-failure exposure via directed cut-vertex articulation scoring ($\text{AP}_{c,\text{directed}}$) and bridge ratios:

$$A(v) = g(\text{AP}_{c,\text{directed}}(v), \text{BR}(v), \text{CDI}(v), w(v)) \quad (6)$$

3. **Reliability Composite (R):** Blends Fault Tolerance and Availability with $\alpha = 0.36$:

$$R(v) = \alpha \cdot FT(v) + (1 - \alpha) \cdot A(v) \quad (7)$$

4. **Maintainability (M):** Blends betweenness centrality (BT), efferent QoS out-degree (w_{out}), enhanced coupling risk, and the Code Quality Penalty (CQP):

$$M(v) = 0.35 \text{BT}(v) + 0.30 w_{\text{out}}(v) + 0.15 \text{CQP}(v) + 0.12 \text{CouplingRisk}_{\text{enh}}(v) + 0.08 (1 - \text{CC}(v)) \quad (8)$$

Stakeholder Harm Projection Matrix. The diagnostic vector $\mathbf{s}_{\text{RM}}(v) = [R(v), M(v)]^T$ projects into ISO/IEC 25019 Quality-in-Use stakeholder harm scores $[H_{\text{Ben}}, H_{\text{Risk}}, H_{\text{Acc}}]^T$ via transformation matrix $\mathbf{M}_{\text{RM} \rightarrow \text{QiU}}$:

$$\mathbf{h}_{\text{QiU}}(v) = \mathbf{M}_{\text{RM} \rightarrow \text{QiU}} \cdot \mathbf{s}_{\text{RM}}(v) = \begin{bmatrix} 0.75 & 0.25 \\ 0.80 & 0.20 \\ 0.60 & 0.40 \end{bmatrix} \begin{bmatrix} R(v) \\ M(v) \end{bmatrix} \quad (9)$$

Composite criticality blends the dimensions under declared weights $(w_R, w_M) = (0.80, 0.20)$ [31]:

$$Q(v) = 0.80 R(v) + 0.20 M(v) \quad (10)$$

Components are categorized into five severity tiers (CRITICAL, HIGH, MEDIUM, LOW, MINIMAL) using adaptive box-plot thresholding ($Q > Q_3 + 1.5 \text{IQR}$ for CRITICAL; $Q_3 < Q \leq \text{upper fence}$ for HIGH).

4 A Catalog of Architectural Anti-Patterns for Publish–Subscribe Systems

4.1 Anti-Patterns vs. Bad Smells in Automated Review

In automated code review, findings fall into two distinct confidence tiers:

- **Architectural Anti-Pattern:** A proven structural pathology known to cause systemic vulnerability or failure cascades, requiring structural intervention (e.g., Single Point of Failure, God Component, Broker Overload).
- **Architectural Bad Smell:** A localized structural heuristic signaling potential technical debt, warranting review and localized refactoring (e.g., Chatty Pair, QoS Mismatch, Orphaned Topic).

Publish–subscribe architectural anomalies leave distinct **topological signatures** in the dependency graph. Because these signatures are computable from static descriptors, automated review bots can flag them before code is deployed.

4.2 Detection Methodology

Detection operates over a 16-element structural metric vector. Twelve metrics are Tier-1 inputs feeding the RM model: Reverse PageRank (RPR), in-degree (DG_{in}), multi-path coupling index (MPCI), topic fan-out criticality (FOC), betweenness (BT), QoS out-degree (w_{out}), clustering coefficient (CC), path complexity (PC), directed articulation score ($AP_{c,directed}$), bridge ratio (BR), connectivity degradation index (CDI), and component weight (w). The Code Quality Penalty (CQP) serves as the thirteenth input, while forward centralities (PageRank, Closeness, Eigenvector) are held at Tier-2 for visualization. Detectors employ **adaptive box-plot thresholds** ($Q_3 + 1.5 IQR$).

4.3 Catalog Overview

Table 2 summarizes the nineteen anti-patterns and smells spanning three severity tiers and four catalog categories.

4.4 Representative Pattern Walkthroughs

1. **SPOF (Single Point of Failure):** A node whose removal disconnects the graph ($AP_{c,directed}(v) > 0$). Detection combines graph connectivity testing with QoS weighting ($QSPOF(v) = AP_c(v) \times w(v)$). Remediation generates container anti-affinity constraints.
2. **GOD_COMPONENT:** An entity exhibiting extreme betweenness and CRITICAL maintainability ($BT(v) > 0.30 \wedge M(v) \in \text{CRITICAL}$). Remediation decomposes pub-sub channels via logical topic splitting.
3. **BROKER_OVERLOAD (Cautionary Benchmark Case):** A broker routing $\geq 2\times$ the median broker’s traffic. *Cautionary finding:* In scenario S05 (two equally overloaded brokers), the within-population relative median rule cannot fire because each broker *is* the median. This highlights the necessity of absolute capacity-aware thresholds (Section 11.2).
4. **CHATTY_PAIR:** Two components maintaining bidirectional high-frequency dependencies across separate topics ($(u \rightarrow v) \wedge (v \rightarrow u)$), representing logical coupling masquerading as pub-sub decoupling.
5. **QOS_MISMATCH:** A publisher offering weaker reliability than the subscriber requires ($w_{pub}(u) < w_{sub}(v) - \tau$). In DDS/ROS 2, this produces silent connection dropouts at runtime without compile-time errors.

Table 2: Publish-Subscribe Architectural Anti-Pattern and Bad Smell Catalog.

Pattern	Severity	Category	Detection Signal	Automated Refactoring
SPOF	CRITICAL	Availability	Articulation point $\wedge$ QoS SPOF score	Anti-affinity reallocation
SYSTEMIC_RISK	CRITICAL	Reliability	Share of CRITICAL components $> 20\%$	Advisory
CYCLE	HIGH	Architecture	Strongly connected component / loop	Advisory
GOD_COMPONENT	CRITICAL	Maintainability	Extreme betweenness $\wedge$ CRITICAL $M(v)$	Logical topic splitting
BOTTLENECK_EDGE	HIGH	Availability	Edge betweenness centrality outlier	Logical topic splitting
BROKER_OVERLOAD	HIGH	Availability	Broker availability $\geq 2 \times$ median load	Anti-affinity reallocation
DEEP_PIPELINE	HIGH	Reliability	Path length $\geq \max(5, P_{75})$	Advisory
TOPIC_FANOUT	MEDIUM	Reliability	Topic subscriber out-degree outlier	Logical topic splitting
CHATTY_PAIR	MEDIUM	Maintainability	Bidirectional weight product $> \tau_{chatty}$	Advisory
QOS_MISMATCH	MEDIUM	Reliability	Pub/Sub QoS-weight gap $> \tau_{qos}$	Advisory (Manual / Relay)
ORPHANED_TOPIC	MEDIUM	Maintainability	Zero in- or out-degree on graph	Advisory
UNSTABLE_INTERFACE	MEDIUM	Maintainability	High <code>CouplingRisk_enh</code> $\wedge$ high $M(v)$	Advisory
BRIDGE_EDGE	HIGH	Availability	Graph-theoretic bridge cut-edge	Advisory
FAILURE_HUB	CRITICAL	Reliability	Reliability outlier $\wedge$ high out-degree	Logical topic splitting
CONCENTRATION_RISK	MEDIUM	Reliability	Top-3 PageRank share > 0.50	Advisory
HUB_AND_SPOKE	MEDIUM	Maintainability	Low clustering $\wedge$ degree > 3	Logical topic splitting
CHAIN	MEDIUM	Architecture	Degree-bounded linear sub-graph	Advisory
ISOLATED	MEDIUM	Architecture	Zero total degree	Advisory
COMPOUND_RISK	CRITICAL	Architecture	Co-occurring SPOF + God/Hub finding	Anti-affinity reallocation

4.5 Detection Validation Methodology

Detection accuracy is validated against an independent discrete-event cascade simulation oracle ($I_{\text{comp}}(v)$) that models cascading message dropouts and queue exhaustion under component removal.

5 Closed-Loop Optimization Objective

The prescriptive refactoring objective computes a graph transformation $\Delta(G)$ producing mutated topology $G' = \Delta(G)$ that minimizes global failure impact subject to a budget constraint:

$$\min_{\Delta} \sum_{v \in V} I_{\Delta(G)}^*(v) \quad \text{subject to} \quad \text{Cost}(\Delta) \leq \mathcal{B} \quad (11)$$

Candidate refactorings are compiled from entities flagged **CRITICAL** or **HIGH** by the Diagnostic Pathway. Rather than relying on an unconstrained heuristic search, SaG-Prescribe enforces a **two-level closed-loop acceptance filter**:

1. **Per-Edit Counterfactual Acceptance Filter:** Each candidate edit δ is applied in isolation to a sandboxed copy of G and simulated across propagation thresholds $\Theta = \{0.1, 0.2, 0.5\}$ and seed set $S = \{42, 123, 456\}$. It is retained if and only if its mean impact reduction exceeds the simulator’s seed noise at every threshold:

$$\overline{\Delta I}_{\theta}(\delta) > \kappa \cdot \sigma_{\text{seed}, \theta}(\delta) \quad \forall \theta \in \Theta \quad (12)$$

where $\kappa = 1.0$ is the noise-rejection margin.

2. **Whole-Policy Acceptance Gate:** The surviving subset of edits Δ_{admitted} is applied jointly to G , and the resulting system is re-evaluated. The whole policy is accepted if and only if the net System Risk Index improves:

$$\Delta \text{SRI} = \text{SRI}_{\text{baseline}} - \text{SRI}_{\text{mutated}} > 0 \quad (13)$$

6 The SaG-Prescribe Prescriptive Pipeline

6.1 Hexagonal Core Architecture

SaG-Prescribe implements a hexagonal ports-and-adapters architecture. The domain service depends on the `IGraphRepository` port:

- **Neo4jRepository:** Bolt-driven graph store for persistent, production deployments.
- **MemoryRepository:** High-speed, in-memory graph repository ensuring thread-safe, isolated sandboxing for counterfactual verification in CI/CD pipelines.

6.2 The Seven Pipeline Stages

1. **Stage 1 (Model Ingestion):** Ingest JSON/YAML Architecture-as-Code descriptors and file-level SCA metrics.
2. **Stage 2 (Topological Projection):** Construct multi-layer graph G_{analysis} and derived `DEPENDS_ON` edges.
3. **Stage 3 (Diagnostic Quality Attribution):** Compute CQP, RM dimensions (FT, A, R, M) , and ISO 25019 harm mapping.

4. **Stage 4 (Anti-Pattern Detection):** Run the 19-pattern detection engine with adaptive box-plot thresholds.
5. **Stage 5 (Candidate Compilation):** Compile candidate refactoring mutations from CRITICAL/HIGH findings.
6. **Stage 6 (Counterfactual Verification):** Execute two-level simulation verification in MemoryRepository.
7. **Stage 7 (Review Interface):** Emit verified refactoring blueprint and CI/CD quality gate verdicts.

6.3 Refactoring Mutation Operators

1. **Logical Topic Splitting (Operator 1):** Targets TOPIC_FANOUT, GOD_COMPONENT, and FAILURE_HUB. Decomposes a multi-publisher topic T into dedicated sub-topics $\{T_a : a \in P(T)\}$, isolating data flows and bounding blast radius.
2. **Physical Anti-Affinity Reallocation (Operator 2):** Targets SPOF, BROKER_OVERLOAD, and COMPOUND_RISK. Generates orchestration anti-affinity constraints to migrate co-located critical components from shared compute host N_{from} to isolated host N_{to} , updating RUNS_ON and CONNECTS_TO edges.
3. **Transport QoS Contract Hardening (Operator 3):** Fires on CRITICAL/HIGH channels operating under volatile contracts (BEST_EFFORT / VOLATILE), upgrading them to RELIABLE and TRANSIENT_LOCAL.

Automation Footprint Disclosure. Exactly five of the nineteen patterns directly trigger an automated mutation operator (SPOF $\rightarrow$ Operator 2; GOD_COMPONENT, BOTTLENECK_EDGE, FAILURE_HUB, HUB_AND_SPOKE $\rightarrow$ Operator 1). Operator 3 triggers from generic criticality tiers. The remaining fourteen patterns provide advisory recommendations for human review.

6.4 Closed-Loop Counterfactual Verification Procedure

Candidate generation and verification are strictly decoupled: candidate refactorings are generated exclusively from G_{analysis} and diagnostic RM attribution, and edits are simulated in isolation on $G_{\text{structural}}$ within MemoryRepository, enforcing complete view independence.

7 DevOps Integration and CI/CD Quality Gating

7.1 Automated Code Review Bot Architecture

To govern architectural quality during continuous evolution, the Diagnostic Pathway is packaged as a lightweight pre-merge review bot. When a pull request modifies system topology or QoS configurations, the bot analyzes the change, executes the 19 anti-pattern detectors, attributes RM debt, and posts structured review comments.

7.2 Regression Semantics: Absolute vs. Delta-Aware Gating

- **Absolute Gating:** Evaluates the entire topology against global thresholds. However, real-world industrial systems carry intentional, risk-accepted legacy debt; blocking builds on pre-existing debt paralyzes development.

- **Delta-Aware Gating (Design):** Compares the PR candidate topology against the target branch merge-base, blocking only *newly introduced* anti-patterns or worsening severities, while honoring a signed **waiver register**.

7.3 Three-Tier Exit-Code Protocol

- **Exit Code 0 (Pass):** No findings above threshold; deployment permitted.
- **Exit Code 1 (Warning):** MEDIUM-severity smells detected; build passes with advisory comments.
- **Exit Code 2 (Block):** CRITICAL or HIGH anti-patterns detected; build fails, blocking merge.

8 Experimental Design

8.1 Research Questions

- **RQ1 (Diagnostic Detection Efficacy & Precision):** How accurately does the Diagnostic Pathway correlate with cascade failure impact, and how effectively does the anti-pattern catalog identify architectural debt without over-flagging?
- **RQ2 (Prescriptive Refactoring Efficacy):** Does the closed-loop prescriptive engine achieve statistically significant reductions in System Risk Index across heterogeneous scenarios?
- **RQ3 (Operator Survival & Contributions):** How do individual refactoring operators survive counterfactual verification across structural regimes?
- **RQ4 (Verification Yield & Compositionality):** What proportion of generated candidates survive verification, and do individually admitted edits compose beneficially?
- **RQ5 (Computational Overhead & CI/CD Feasibility):** What is the execution latency of diagnostic gating and prescriptive verification across system scales?

8.2 Benchmark Scenarios

We evaluate eight benchmark scenarios spanning distinct architectural domains (Table 3). Scenarios S01–S08 evaluate detection; S01–S07 evaluate prescription (S08 excluded as a minimal regression smoke test).

Table 3: Benchmark Scenario Scales and Topological Dimensions.

Scenario	Applications	Libraries	Topics	Brokers	Nodes	Total Edges ($ E $)
S01 Autonomous Vehicle	80	20	40	4	8	797
S02 IoT Smart City	200	10	80	6	30	1322
S03 Financial Trading	60	18	35	5	6	580
S04 Healthcare	50	12	25	3	8	400
S05 Hub-and-Spoke	70	25	30	2	12	797
S06 Microservices Mesh	90	30	45	6	15	680
S07 Hyper-Scale Enterprise	300	50	120	10	40	3245
S08 Tiny Regression	12	4	8	2	3	48

8.3 Experimental Protocol and Metrics

- **Detection Metrics:** Spearman rank correlation $\rho(Q, I_{\text{comp}})$ (pooled and stratified), precision, recall, F_1 , and Top-5 overlap against simulation ground truth I_{comp} . Baselines: betweenness and degree centrality.
- **Prescriptive Metrics:** System Risk Index ($\text{SRI} = 0.5(1 - H_R) + 0.5(1 - H_M)$), $\Delta\text{SRI} = \text{SRI}_{\text{baseline}} - \text{SRI}_{\text{mutated}}$, per-operator candidate generation counts, admitted edit counts, and per-edit survival rates.
- **Verification Parameters:** $\kappa = 1.0$, $\Theta = \{0.1, 0.2, 0.5\}$, $S = \{42, 123, 456\}$.

9 Empirical Results

9.1 Diagnostic Detection Efficacy and Precision (RQ1)

Table 4 presents detection validation at the application layer across all eight scenarios.

Table 4: Diagnostic Detection Validation (App Layer) against Simulation Ground Truth $I_{\text{comp}}(v)$.

Scenario	n	$\rho(Q, I)$	F_1	Top-5	Catalog P	Catalog R	Implicated %	Gate (s)
S01 Autonomous Vehicle	80	0.457	0.600	0.40	0.244	0.950	97.5%	0.59
S02 IoT Smart City	200	0.718	0.780	0.40	0.310	0.880	71.0%	0.56
S03 Financial Trading	60	0.365	0.667	1.00	0.237	0.933	98.3%	0.40
S04 Healthcare	50	0.451	0.538	0.60	0.213	0.769	94.0%	0.16
S05 Hub-and-Spoke	70	0.466	0.611	0.80	0.257	1.000	100.0%	0.92
S06 Microservices Mesh	90	0.405	0.565	0.40	0.261	1.000	97.8%	0.35
S07 Hyper-Scale Enterprise	300	0.480	0.627	0.20	0.251	1.000	99.7%	20.98
S08 Tiny Regression	12	0.538	0.333	0.60	0.250	1.000	100.0%	0.01
Mean	—	0.485	0.590	0.550	0.253	0.942	94.8%	3.00

Key Diagnostic Findings.

1. **Ranking Efficacy:** Mean Spearman correlation is $\rho = 0.485$ (range 0.365–0.718). Comparing against single-metric baselines, betweenness centrality achieves $\rho = 0.435$, while **degree centrality alone achieves** $\rho = 0.519$. The value of the RM Diagnostic Pathway lies not in superior scalar ranking, but in **explainable quality attribution**—routing structural flaws to specific stakeholder roles (Table 1), which a scalar degree count cannot achieve.
2. **Catalog Screening Behavior:** The anti-pattern catalog functions as a sensitive screen: mean recall is 0.942, while precision is 0.253. The catalog implicates a mean of 94.8% **of components** at CRITICAL/HIGH severity. This high base-rate confirms that uncalibrated global box-plot thresholds over-flag in dense topologies, underscoring the urgent necessity of delta-aware CI/CD gating (Section 7).
3. **Simpson’s Paradox in Node Pooling:** At the system layer (pooling all five node types), the pooled correlation drops to $\rho = 0.028$, while per-type strata show robust positive correlations (Application 0.503, Broker 0.395, Node 0.142). This documents a severe Simpson’s paradox resulting from between-type score offsets, proving that stratified evaluation is methodologically essential for heterogeneous software graphs.

9.2 Prescriptive Refactoring Efficacy (RQ2)

Table 5 presents prescriptive refactoring results under per-edit counterfactual verification across all seven benchmark scenarios.

Table 5: Prescriptive Refactoring Performance under Per-Edit Counterfactual Verification.

Scenario	Base SRI	Mut SRI	Δ SRI	Candidates	Admitted	Rejected	Admit %
S01 Autonomous Vehicle	0.3246	0.3160	+0.0086	156	45	111	28.8%
S02 IoT Smart City	0.3749	0.3288	+0.0461	391	297	94	76.0%
S03 Financial Trading	0.3409	0.3301	+0.0108	125	22	103	17.6%
S04 Healthcare	0.3394	0.3285	+0.0109	99	36	63	36.4%
S05 Hub-and-Spoke	0.3223	0.3108	+0.0115	127	107	20	84.3%
S06 Microservices Mesh	0.3293	0.3196	+0.0097	163	142	21	87.1%
S07 Hyper-Scale Enterprise	0.3283	0.3112	+0.0171	528	479	49	90.7%
Total / Overall	—	—	—	1589	1128	461	71.0%

Key Prescriptive Findings.

- Statistically Significant Improvement:** Every scenario achieves a net positive Δ SRI. A Wilcoxon signed-rank test against zero yields $W = 0, p = 0.0156 (n = 7)$, confirming statistically significant risk reduction across all evaluated domains.
- Substantial Yield:** Across 1589 generated candidates, **1128 edits (71.0%)** survive counterfactual verification.

9.3 Operator Survival and Contributions (RQ3)

Table 6 breaks down candidate generation and admission rates by refactoring operator.

Table 6: Candidate Generation vs. Verified Admission by Refactoring Operator.

Refactoring Operator	Candidates	Admitted	Survival Rate	Primary Structural Target
Physical Anti-Affinity Reallocation	1188	923	77.7%	Single points of failure, co-located components
Transport QoS Contract Hardening	69	40	58.0%	Volatile transport contracts over unreliable links
Logical Topic Splitting	332	165	49.7%	High-fan-out message channels
Total	1589	1128	71.0%	—

Anti-affinity reallocation demonstrates the highest survival rate (77.7%), achieving near-100% survival in dense topologies (408/409 in Enterprise, 123/123 in Microservices). Transport QoS hardening achieves 58.0% survival where triggered, while topic splitting achieves 49.7%.

9.4 Verification Yield and Compositionality (RQ4)

- Informative Rejection:** All 461 rejected edits record the binding propagation threshold θ where impact reduction failed to clear $\kappa\sigma_{\text{seed}}$, providing transparent review feedback.
- Compositional Dynamics:** While every scenario achieves net positive system-level Δ SRI, two scenarios (S03 Financial Trading and S06 Microservices Mesh) exhibit slight negative mean per-component changes among accepted edits. This demonstrates that individually verified edits can interact sub-additively, validating the necessity of the second-level whole-policy gate.

9.5 Computational Overhead and CI/CD Feasibility (RQ5)

1. **Diagnostic Review Gate Latency:** The complete diagnostic pipeline (graph construction, CQP computation, RM attribution, and 18 active anti-pattern detectors) executes in **0.01 s to 20.98 s** (Table 4). The eighteen detectors run in under **0.2 s total**; computational overhead is dominated by base metric computation.
2. **DEEP_PIPELINE Bounding:** The unconstrained DEEP_PIPELINE detector attempts exhaustive path enumeration, generating 247,761 findings on tiny fixtures and hanging on larger graphs. Excluding this detector restores sub-second gate execution.
3. **Prescriptive Verification Latency:** The full 7-scenario prescriptive sweep (14,301 simulation sweeps) completed in **2 h 47 min** using multi-core process parallelism (20 workers), establishing its viability for nightly or on-demand CI/CD refactoring runs.

10 Discussion and Threats to Validity

10.1 Diagnostic Explainability vs. Threshold Calibration

The Diagnostic Pathway successfully provides deterministic, role-specific quality attribution (Reliability Engineer vs. SRE vs. Architect), translating abstract topological risk into actionable review decisions. However, its current box-plot thresholds over-flag in dense graphs (94.8% implicated). Threshold recalibration and delta-aware gating are essential for production CI/CD deployment.

10.2 The Verify-Before-Recommend Discipline

Counterfactual verification proves essential: an unverified engine would have applied 461 harmful or ineffective mutations (29.0%). By filtering mutations against cascade noise, SaG-Prescribe ensures that recommended refactorings reliably harden system architecture.

10.3 Threats to Validity

- **Construct Validity:** Both detection and verification utilize a discrete-event cascade simulator. We mitigate simulator dependency by grounding operators in established distributed systems dependability patterns (anti-affinity scheduling, topic isolation, QoS hardening).
- **Internal Validity:** Decoupled in-memory repositories ensure no circular data leakage between diagnostic candidate generation and counterfactual simulation.
- **External Validity:** Scenarios are synthetic domain models; validation on harvested industrial cyber-physical codebases remains high-priority future work.

11 Conclusion and Future Work

11.1 Conclusion

This paper presented **SaG-Prescribe**, bringing the **Diagnostic Pathway** to the forefront of automated code review and software quality evaluation for distributed publish-subscribe systems. By integrating code-level SCA metrics (CQP), multi-layer heterogeneous dependency modeling, hierarchical ISO 25010/25019 RM quality attribution, and formal anti-pattern detection with counterfactually verified refactoring, SaG-Prescribe closes the Architecture-Code Gap. Empirical results confirm that diagnostic review gating executes in sub-second to 20s latency, while

counterfactual verification reliably admits 71.0% of refactoring candidates, delivering statistically significant resilience improvements across all evaluated domains.

11.2 Future Work

1. **Detection Threshold Recalibration:** Sweep percentile fences to optimize precision/recall balance.
2. **Path-Bounded DEEP_PIPELINE:** Restrict path enumeration to bound execution time.
3. **Delta-Aware CI/CD Gating:** Implement merge-base diffing and waiver registers.
4. **Combinatorial Subset Verification:** Explore greedy forward-selection to optimize multi-edit composition.
5. **Dynamic κ Estimation:** Adapt noise margins based on empirical simulator variance.
6. **Per-Candidate Latency Profiling:** Instrument fine-grained verification timing.
7. **Operator Attribution Ablation:** Isolate per-operator Δ SRI contributions.
8. **Catalog Extension for Hybrid Architectures:** Incorporate REST/gRPC hybrid microservice patterns.
9. **Industrial Telemetry Replication:** Validate against live ROS 2 and DDS production traces.
10. **LLM-Assisted Pull-Request Synthesis:** Integrate generative code models to automatically draft pull requests implementing verified refactoring blueprints.

Declarations

Funding: This work was supported in part by the Scientific and Technological Research Council of Turkey.

Competing Interests: The authors declare that they have no competing financial or non-financial interests that are directly or indirectly related to the work submitted for publication.

Data Availability: Replication code, scenario topologies, seeds, and execution logs are reproducible via the repository harness:

```
PYTHONPATH=. python reproduce/detection_validation.py --layer app \  
--output results/detection_validation_app.json  
PYTHONPATH=. python reproduce/run_prescribe_all.py --kappa 1.0
```

Ethics Approval: Not applicable.

References

- [1] R. Tufano, L. Pascarella, M. Tufano, D. Poshypanyk, and G. Bavota, “Towards automating code review activities,” *IEEE Transactions on Software Engineering*, vol. 48, no. 8, pp. 3156–3173, 2022.

- [2] S. Lee, H.-S. Park, J. Chae, and K.-J. Park, “Probabilistic latency analysis of the data distribution service in ros 2,” *IEEE Transactions on Industrial Informatics*, vol. 21, no. 3, pp. 2415–2426, 2025.
- [3] S. Lee, J. Kang, and K.-J. Park, “Dependency chain analysis of ros 2 dds qos policies: From lifecycle tutorial to static verification,” *IEEE Internet of Things Journal*, vol. 12, no. 4, pp. 3890–3901, 2025.
- [4] M. Fowler, *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 1999.
- [5] W. H. Brown, R. C. Malveau, H. W. McCormick, and T. J. Mowbray, *AntiPatterns: Refactoring Software, Architectures, and Projects in Crisis*. John Wiley & Sons, 1998.
- [6] G. Suryanarayana, G. Samarthayam, and T. Sharma, *Refactoring for Software Design Smells: Managing Technical Debt*. Morgan Kaufmann, 2014.
- [7] D. Taibi, V. Lenarduzzi, and C. Pahl, “Microservices anti-patterns: A taxonomy,” in *Microservices: Science and Engineering*. Springer, 2020, pp. 211–228.
- [8] M. Harman, S. A. Mansouri, and Y. Zhang, “Search-based software engineering: Trends, techniques and applications,” *ACM Computing Surveys*, vol. 45, no. 1, pp. 1–61, 2012.
- [9] A. Aleti, B. Buhnova, L. Grunske, A. Koziolk, and I. Meedeniya, “Software architecture optimization methods: A systematic literature review,” *IEEE Transactions on Software Engineering*, vol. 39, no. 5, pp. 658–683, 2013.
- [10] ISO/IEC, “Iso/iec 25010:2023 systems and software engineering — systems and software quality requirements and evaluation (square) — product quality model,” International Organization for Standardization, Tech. Rep., 2023.
- [11] —, “Iso/iec 25019:2023 systems and software engineering — systems and software quality requirements and evaluation (square) — quality-in-use model,” International Organization for Standardization, Tech. Rep., 2023.
- [12] Authors, “Software-as-a-graph: A static system analysis framework for pre-deployment quality gating and failure simulation of publish-subscribe middleware,” *Journal of Systems and Software*, 2026, under review.
- [13] S. Pallickara, H. Bulut, and G. Fox, “Fault-tolerant reliable delivery of messages in distributed publish/subscribe systems,” in *Proc. 4th IEEE International Conference on Autonomic Computing (ICAC)*, 2007, pp. 12–21.
- [14] T. Chang, S. Duan, H. Meling, S. Peisert, and H. Zhang, “P2s: A fault-tolerant publish/subscribe infrastructure,” in *Proc. 8th ACM International Conference on Distributed Event-Based Systems (DEBS)*, 2014, pp. 198–207.
- [15] G. Wang, J. Koshy, S. Subramanian, K. Paramasivam, M. Zadeh, N. Narkhede, J. Rao, J. Kreps, and J. Stein, “Building a replicated logging system with apache kafka,” *Proceedings of the VLDB Endowment*, vol. 8, no. 12, pp. 1654–1655, 2015.
- [16] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. Clement, D. Drain, D. Jiang, D. Tang, G. Li, L. Zhou, D. Jiang, M. Zhou, and N. Duan, “Codexglue: A machine learning benchmark dataset for code understanding and generation,” in *Proc. NeurIPS Datasets and Benchmarks*, 2021.
- [17] T. Menzies and F. Rahman, “Bad smells in software analytics papers,” *IEEE Software*, vol. 35, no. 4, pp. 100–103, 2018.

- [18] C. Richardson, *Microservices Patterns: With Examples in Java*. Manning Publications, 2018.
- [19] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, “A fast and elitist multiobjective genetic algorithm: Nsga-ii,” *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.
- [20] E. AlOmar, M. W. Mkaouer, and A. Ouni, “Can refactoring be self-affirmed? an exploratory study on how developers document their refactorings,” *Empirical Software Engineering*, vol. 26, no. 6, p. 121, 2021.
- [21] G. Bavota, B. De Carluccio, R. Oliveto, M. Di Penta, and A. De Lucia, “When does a refactoring induce bugs? an empirical study,” in *Proc. IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM)*, 2012, pp. 104–113.
- [22] A. Ouni, M. Kessentini, H. Sahraoui, and M. S. Hamdi, “Search-based web service refactoring using quality of service and code quality metrics,” *IEEE Transactions on Services Computing*, vol. 10, no. 4, pp. 636–649, 2017.
- [23] C. White, S. Agarwal, and H. Zhang, “Toward an understanding of large language models for code refactoring,” *IEEE Software*, vol. 41, no. 2, pp. 48–56, 2024.
- [24] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, “Large language models for software engineering: A systematic literature review,” *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 5, pp. 1–68, 2024.
- [25] A. M. Al-Kaswan, T. Ahmed, M. Izadi, A. A. Sawant, P. Devanbu, and A. van Deursen, “Automatic refactoring using large language models: A study on extract method and rename,” *Empirical Software Engineering*, vol. 29, no. 4, p. 98, 2024.
- [26] L. C. Freeman, “A set of measures of centrality based on betweenness,” *Sociometry*, vol. 40, no. 1, pp. 35–41, 1977.
- [27] A. Bakhtin, M. Esposito, V. Lenarduzzi, and D. Taibi, “Network centrality as a new perspective on microservice architecture,” in *Proc. IEEE International Conference on Software Architecture (ICSA)*, 2025, pp. 72–83.
- [28] D. H. M. Falci, O. A. Gomes, and F. S. Parreiras, “Complex networks analysis for software architecture: An hibernate call graph study,” *IEEE Access*, vol. 6, pp. 62 145–62 155, 2018.
- [29] C. Y. Baldwin and K. B. Clark, *Design Rules, Volume 1: The Power of Modularity*. MIT Press, 2000.
- [30] C. J. Colbourn, *The Combinatorics of Network Reliability*. Oxford University Press, 1987.
- [31] T. L. Saaty, *The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation*. McGraw-Hill, 1980.